DarkSolver: Formal Verification of EVM Business-Logic Invariants via Bounded `QF_ABV` Symbolic Execution

DarkSolver Research Notes

March 12, 2026

Abstract

DarkSolver is a simulation-first auditing primitive for Ethereum Virtual Machine (EVM) contracts. Its public repository centers on a bounded symbolic execution engine, a target-hydration layer, protocol-aware exploit objectives, invariant-gated admissibility checks, and a replay-oriented verification layer for curating concrete evidence. This document expands the repository's brief technical note into a whitepaper-style description of the implemented analysis model.

The key design choice is to encode EVM business-logic search problems into Z3's quantifier-free array/bit-vector fragment `QF_ABV`, preserving EVM word width while remaining tractable enough for protocol-specific objective families. DarkSolver does not claim complete formal verification of arbitrary contracts. Instead, it exposes a bounded, fail-closed search procedure: uncertain branches are terminated rather than permissively approximated, unresolved hydration is treated conservatively, and replay-style evidence is separated from symbolic satisfiability so downstream reporting policy can remain strict.

This paper formalizes the state domain, small-step transition relation, storage and Keccak abstractions, target hydration, objective synthesis, invariant gates, execution-status semantics, and replay-verification interface. It also states the repository's actual guarantees and non-guarantees. The result is intended to be legible to grant reviewers, formal-methods researchers, and engineers comparing the math to the current implementation surface.

Notation Summary

Symbol	Domain	Meaning
B	$(\{0,1\}^8)^*$	Target bytecode image.
Γ	$\mathcal{E}$	Chain-level analysis environment: RPC, chain id, base block context, and runtime bounds.
Θ	contextual record	Hydrated target context: address, bytecode metadata, selector slices, dependency snapshots, and zero-state classification.
Σ	$\mathbb{S}$	Symbolic machine state used by the opcode semantics.
pc	$\mathbb{N}$	Program counter in the current bytecode image.
gas	$\mathbb{W}_{256}$	Remaining gas budget or gas-derived symbolic term.

Symbol	Domain	Meaning
stack	$(\mathbb{W}_{256})^*$	EVM stack represented as a bounded list of 256-bit words.
mem	$\mathbb{N} \to \{0,1\}^8$	Byte-addressable symbolic memory.
stor	$\mathbb{W}_{256} \to \mathbb{W}_{256}$	Raw storage-slot map, augmented by structured traces.
π	Boolean formula	Accumulated path predicate over `QF_ABV` terms.
$\mathcal{J}$	nested write log	Side-effect journal used for snapshots, rollback, and replay-relevant accounting.
$\mathcal{T}$	finite sequence	Sequence of SHA3 trace tuples recorded during symbolic execution.
res	finite map	Pair-addressed reserve summaries used by invariant gates.
δ_B	$\mathbb{S} \to \mathcal{P}(\mathbb{S} \cup \{\bot_{\mathsf{fail}}, \bot_{\mathsf{halt}}\})$	Small-step transition relation induced by bytecode B.
$\mathsf{Hydrate}$	$(\mathcal{A}, \Gamma) \to \Theta$	Target hydration function.
$\mathcal{O}$	finite family	Objective constructors instantiated from the objective catalog.
Gate	predicate	Triple-gate invariant filter: solvency, price sanity, and reserve-product consistency.
Replay	witness $\times$ env $\to$ report	Fork-based replay and valuation layer for witness curation.
Status	$\{\mathsf{SAT}, \mathsf{UNSAT}, \mathsf{TIMEOUT}, \mathsf{PANIC}\}$	Per-objective execution outcome used by the detailed runner.
$\mathsf{Reportable}$	policy predicate	Downstream policy that decides whether a symbolic witness plus replay report is strong enough for publication.

1 Introduction and Problem Statement

Pattern-based smart-contract analyzers are useful for broad screening, but they are poorly matched to protocol-specific business logic. A reserve-product manipulation opportunity, a share-rounding griefing path, or a proxy-initialization race is usually not recognizable from syntax alone. The question of interest is semantic: does there exist a bounded sequence of EVM actions such that a protocol-specific invariant is violated while all machine-level and selected economic constraints remain feasible?

DarkSolver addresses that question with a symbolic, simulation-first workflow. The repository's public path is organized around single-target analysis: hydrate a contract and a bounded amount of surrounding state, derive a curated family of exploit objectives, solve those objectives in independent Z3 contexts, and emit witness parameters suitable for reproduction or later replay-oriented curation. The design is intentionally not a theorem prover for arbitrary EVM programs. It is a bounded search primitive whose emphasis is on *defensible exploit evidence* rather than exhaustive language-level proof obligations.

At a high level, the repository materializes the following evidence relation:

$$\mathcal{E}_{\text{repo}}(B,\Gamma) = \left\{ (o,m,\rho) \;\middle|\; \begin{array}{l} o \in \mathcal{O}(\Gamma), \\ m \models \Phi_o(B,\Gamma), \\ \mathsf{Gate}(m,\Gamma), \\ \rho = \mathsf{Replay}(m,\Gamma), \\ \mathsf{Reportable}_{\Pi}(m,\rho) \end{array} \right\}, \tag{1.1}$$

where o is an instantiated objective, m is a satisfying model or witness candidate, Φ_o is that objective's symbolic query, Gate is the global admissibility filter, ρ is a replay/valuation report, and Π is an external reporting policy. Equation (1.1) is stated at the repository level rather than as a promise that every public CLI command executes every stage unconditionally. In the current codebase, symbolic solving is the primary public command surface, while replay validation lives as a verifier layer and companion tooling used for report curation.

This paper makes four concrete contributions.

1. It gives a mathematically explicit state-transition model for the symbolic EVM execution used in the repository.
2. It defines the `QF_ABV` encodings and approximation boundaries that make the search practical, especially for width-sensitive arithmetic and Keccak-derived storage.
3. It formalizes the repository's protocol-aware objective and invariant pipeline, including the fail-closed execution-status semantics of the parallel runner.
4. It states what the implementation actually guarantees, what it only approximates, and what remains out of scope.

The rest of the document follows the public workflow: system model, symbolic semantics, solver encoding, hydration, objectives, global gates, parallel execution, replay validation, correctness posture, boundedness, and limitations.

2 System Model and Threat Model

2.1 Environment

DarkSolver analyzes one contract at a time against a chain-aware environment

$$\Gamma = \langle \mathsf{chain_id}, \mathsf{rpc}, b_0, t_0, f_0, \mathcal{H}, \Theta \rangle, \tag{2.1}$$

where $\mathsf{chain_id}$ is the execution chain identifier, rpc is the hydration/replay endpoint family, b_0 is the base block number, t_0 is the base timestamp, f_0 is the base fee, $\mathcal{H}$ is the set of runtime bounds and feature toggles, and Θ is the hydrated target context defined later in section 7.

The environment is not static metadata alone. It includes the bounded pre-analysis choices that materially influence tractability: proxy-resolution depth, storage-scan limits, selector slicing, deep-scan enablement, objective caps, and solver timeouts. These are not implementation noise. They define the effective analysis regime under which claims in this paper should be read.

2.2 Adversary Model

The public workflow assumes an adversary who may choose bounded calldata sequences, attacker-controlled addresses, bounded flash-loan amounts, and optional inter-call block offsets when an objective family supports multi-block modeling. The adversary does *not* control the target bytecode, the base fork state, or the raw EVM semantics. The attacker model is therefore:

1. EVM-valid calldata can be constructed for selected entry points.
2. Caller identity and economically relevant input amounts can be symbolic.
3. Objective families may encode protocol-specific attacker actions such as swap sequencing, flash-loan legs, governance votes, or reentrancy-triggering call patterns.
4. State evolution remains bounded by the objective-specific search template and the symbolic engine's transition semantics.

This is narrower than "arbitrary omniscient adversary" reasoning and stronger than pure linting. DarkSolver is explicitly targeted at *bounded exploit search*.

2.3 Out-of-Scope Claims

Three non-claims should be explicit from the beginning.

1. The repository does not prove semantic safety for all inputs or all future chain states.
2. The repository does not model Keccak cryptographically; it models structured equality and dependence through traced uninterpreted functions.
3. The public workflow does not imply that every satisfiable symbolic witness is automatically exploitable on-chain; replay, valuation, and human review remain separate filters.

Assumption 2.1 (Bounded objective search)**.** *Every objective instantiated by the public workflow commits to a finite search template: a finite selector set, finite step family, finite structural bounds on hydration, and a finite solver budget.*

Remark 2.2. Definition 2.1 is not a proof obligation invented for exposition. It mirrors the implementation directly: objective counts are capped, proxy expansion is bounded, storage scans are bounded, and solver contexts are configured with explicit timeout and resource limits.

3 EVM State Domain

3.1 Machine state

DarkSolver's symbolic engine works over a state space $\mathbb{S}$ whose elements are tuples

$$\Sigma = \langle \mathsf{pc}, \mathsf{gas}, \mathsf{stack}, \mathsf{mem}, \mathsf{stor}, \mathsf{env}, \pi, \mathcal{J}, \mathcal{T}, \mathsf{res}, \mathsf{bal}, \mathsf{dead} \rangle. \tag{3.1}$$

Each component has an implementation-grounded role.

- $\mathsf{pc} \in \mathbb{N}$ is the current program counter.
- $\mathsf{gas} \in \mathbb{W}_{256}$ is the gas-related symbolic word.
- $\mathsf{stack} \in (\mathbb{W}_{256})^*$ is the EVM stack.
- $\mathsf{mem} : \mathbb{N} \to \{0,1\}^8$ is byte-addressable memory.
- $\mathsf{stor} : \mathbb{W}_{256} \to \mathbb{W}_{256}$ is the raw storage-slot map.
- env contains caller, value, block, and chain context.
- π is the path predicate accumulated so far.
- $\mathcal{J}$ is a nested side-effect journal used for snapshots and rollback.
- $\mathcal{T}$ stores SHA3 trace tuples from memory hashing sites.
- res maps AMM pairs to manipulated reserve expressions used by invariant gates.
- bal records balance overrides, including attacker-side symbolic accounting.
- dead is the set of statically identified PCs that the engine may treat as dead-end control points.

The state is richer than a textbook EVM configuration because the repository is not only interpreting opcodes. It is maintaining analysis-specific summaries needed for downstream admissibility checks and curation. The manipulated reserve map and SHA3 traces are the two clearest examples.

3.2 Target context

Symbolic execution is parameterized by a target context

$$\Theta = \langle \mathsf{target}, \mathsf{zero_state}, \mathsf{account_info}, \mathsf{storage_slots}, \mathsf{sel}, \mathsf{nft_selectors}, \mathsf{dead}, \mathsf{dep} \rangle. \tag{3.2}$$

The distinction between Σ and Θ matters. Σ is per-path symbolic state. Θ is the hydrated contract summary shared across objectives: resolved bytecode facts, scanned selector slices, storage snapshots, dependency contexts, and the strict zero-state marker used to avoid phantom-liquidity false positives.

Definition 3.1 (Zero-state marker)**.** *A hydrated target is marked* ZERO_STATE *iff its bytecode is empty, its remote ETH balance is zero or non-existent, its storage scan completed successfully, and the scan returned no non-zero storage slots.*

This is a deliberately strict definition. Missing storage data does not entitle the analysis to assume emptiness.

3.3 Witnesses and reports

An objective-level symbolic witness is a bounded structured artifact

$$w = \langle \mathsf{loan}, \mathsf{legs}, \mathsf{steps}, \mathsf{offsets}, \mathsf{meta} \rangle, \tag{3.3}$$

where loan is the principal amount, legs is the flash-loan schedule, steps is the ordered calldata path, $\mathsf{offsets}$ is the optional block-offset vector, and meta is objective-specific metadata such as estimated profit or objective-local annotations.

Replay and report curation operate on witnesses, not on arbitrary solver models. This is an important boundary: the symbolic engine may reason over internal terms that never need to be surfaced, while reports consume a finite witness encoding that can be replayed or explained.

4 Small-Step Transition Semantics

4.1 Transition relation

For a fixed bytecode image B, DarkSolver defines a bounded symbolic transition relation

$$\delta_B : \mathbb{S} \to \mathcal{P}(\mathbb{S} \cup \{\bot_{\mathsf{fail}}, \bot_{\mathsf{halt}}\}). \tag{4.1}$$

The codomain of δ_B is deliberately set-valued. A single opcode may produce:

- no successor, for example on terminal failure;
- exactly one successor, for deterministic arithmetic or memory updates;
- multiple successors, for branch-producing or path-splitting instructions.

The repository adopts a fail-closed semantics: if the engine cannot justify a permissive symbolic branch with the information available, it produces $\bot_{\mathsf{fail}}$ rather than a fabricated successor.

4.2 Representative rules

The implementation spans many opcodes, but a small set of rules captures the style of the semantics.

Arithmetic. For `ADD`, if $B[\mathsf{pc}] = \texttt{ADD}$ and the top stack words are x and y,

$$\frac{\mathsf{stack} = x :: y :: \mathsf{stack}'}{\delta_B(\Sigma) \ni \Sigma'} \quad \text{with} \quad \Sigma' = \Sigma[\mathsf{pc} \mapsto \mathsf{pc} + 1,\ \mathsf{stack} \mapsto \mathrm{ADD}_{256}(x, y) :: \mathsf{stack}']. \tag{4.2}$$

The arithmetic operator is defined in section 5; crucially, it remains in 256-bit space.

Storage loads and writes. For `SLOAD`, the successor reads from the raw slot map stor and may additionally consult structured projections inferred from SHA3 traces. For `SSTORE`, the successor updates the raw map and journals the previous value so later rollback or replay-sensitive accounting remains well-defined.

Conditional control flow. For `JUMPI`, the engine may split the state when the branch guard g is symbolically satisfiable both true and false:

$$\delta_B(\Sigma) \supseteq \{\Sigma[\mathsf{pc} \mapsto \mathsf{dst}, \pi \mapsto \pi \wedge g], \Sigma[\mathsf{pc} \mapsto \mathsf{pc} + 1, \pi \mapsto \pi \wedge \neg g]\}\,. \tag{4.3}$$

If either target validation, stack discipline, or guard construction fails in a way that the engine cannot justify, the branch is terminated via $\bot_{\mathsf{fail}}$ rather than widened.

SHA3 trace capture. For hashing over memory windows whose preimage words can be recovered, the successor not only pushes a hash term but also appends a trace tuple to $\mathcal{T}$. This side-channel is semantically important because later storage reasoning depends on it:

$$\Sigma' = \Sigma[\mathcal{T} \mapsto \mathcal{T} \parallel \langle p, h, n, \mathsf{pc} \rangle,\ \mathsf{stack} \mapsto h :: \mathsf{stack}'] . \tag{4.4}$$

Calls. The call semantics in the repository are bounded and analysis-oriented. Call instructions are not treated as unrestricted arbitrary-state transformers. Instead, the engine threads call-context data through symbolic state and objective-specific logic, while the hydration and verification layers provide the concrete target/dependency data needed to prevent unconstrained hallucinations.

Terminal states. The relation may yield $\bot_{\mathsf{halt}}$ for explicit terminal execution (stop, return, revert, halt) and $\bot_{\mathsf{fail}}$ for modeling failure, malformed state, or fail-closed rejection. These are semantically distinct: $\bot_{\mathsf{halt}}$ is an execution outcome, while $\bot_{\mathsf{fail}}$ is an analysis-policy outcome.

4.3 Fail-closed philosophy

DarkSolver's branch semantics intentionally prefer false negatives over unjustified false positives. This is visible beyond control flow:

- malformed ABI decoding is not widened into "some calldata" success;
- hydration uncertainty is not silently replaced by permissive concrete defaults;
- worker failure in the parallel runner aborts the overall run rather than preserving partial findings.

The fail-closed principle is one of the repository's strongest alignment points between code and mathematics, so it reappears throughout later sections.

5 SMT Encoding

5.1 Bit-vector domain

The core solver domain is Z3's `QF_ABV` fragment. Let

$$\mathbb{W}_{256} = \{0,1\}^{256}, \qquad \mathbb{W}_{512} = \{0,1\}^{512}.$$

All symbolic EVM values are represented as fixed-width bit-vectors, not mathematical integers. This matters because EVM arithmetic is modular by definition.

Representative arithmetic encodings are:

$$\mathrm{ADD}_{256}(x, y) = (x + y) \bmod 2^{256}, \tag{5.1}$$

$$\mathrm{SUB}_{256}(x, y) = (x - y) \bmod 2^{256}, \tag{5.2}$$

$$\mathrm{MUL}_{256}(x, y) = (x \cdot y) \bmod 2^{256}. \tag{5.3}$$

When modular wrap-around would invalidate an economic comparison, the implementation lifts operands into a larger exact domain before multiplying or comparing:

$$\mathrm{lift}_{256\to512}(x) = \mathsf{zeroExtend}_{256}(x), \qquad \mathrm{MUL}_{512}(x,y) = \mathrm{lift}_{256\to512}(x)\cdot\mathrm{lift}_{256\to512}(y). \tag{5.4}$$

The invariant layer uses this exact lift to avoid accidental acceptance of wrapped reserve products or ratio numerators.

5.2 Concrete-to-symbolic conversion

The repository's concrete $U256$-to-bit-vector conversion avoids lossy string-round trips. A 256-bit word is decomposed into four 64-bit limbs and then concatenated:

$$\mathsf{word}(u) = \mathrm{concat}(w_0, w_1, w_2, w_3), \tag{5.5}$$

where $w_i \in \{0,1\}^{64}$ are the big-endian limbs of u. This detail is easy to dismiss as implementation trivia, but it is mathematically relevant: it preserves a total, width-correct embedding of concrete words into the symbolic domain.

5.3 Path formulas

The path predicate π is a formula in `QF_ABV` over machine words, selector constants, calldata slices, balance terms, and objective-specific auxiliary variables. Path extension is conjunction:

$$\pi_{k+1} = \pi_k \wedge \varphi_k. \tag{5.6}$$

If $\pi_k \wedge \varphi_k$ is provably infeasible, the corresponding branch is eliminated.

5.4 Address embedding

Although EVM addresses are 160-bit values, the symbolic engine embeds them into 256-bit words for uniformity:

$$\mathrm{addr_bv}(a) = \mathsf{word}(\mathrm{be}_{256}(a)), \tag{5.7}$$

where $\mathrm{be}_{256}(a)$ denotes the 256-bit big-endian word formed by zero-extending the 20-byte address. This keeps address equality, hashing inputs, and storage-index derivations inside the same fixed-width domain.

5.5 Solver configuration

The current public build configures each solver context with deterministic seeds and explicit budgets:

$$\mathsf{timeout} = 60{,}000 \text{ ms}, \tag{5.8}$$

$$\mathsf{rlimit} = 200{,}000{,}000, \tag{5.9}$$

$$\mathsf{logic} = \texttt{QF_ABV}, \tag{5.10}$$

$$\mathsf{random_seed} = 42. \tag{5.11}$$

The configuration also enables partial models. The intent is not to make satisfiability deterministic in every operational sense, but to remove avoidable nondeterminism from the solver interface itself.

Remark 5.1. The symbolic engine explicitly avoids a quantifier-heavy formulation. The current code comments note that global quantifiers were removed for tractability; the math in this paper therefore reflects a bounded quantifier-free regime, not a hidden higher-order proof system.

6 Storage and Keccak Abstraction

6.1 Raw and structured storage

DarkSolver maintains both a raw storage map and a structured shadow view inferred from hash traces. Formally, let

$$\mathsf{stor} = \mathsf{stor}_{\mathrm{raw}} \cup \mathsf{stor}_{\mathrm{shadow}}, \tag{6.1}$$

where $\mathsf{stor}_{\mathrm{raw}}$ maps concrete 256-bit slots to 256-bit words and $\mathsf{stor}_{\mathrm{shadow}}$ is a derived set of structured projections such as flat mappings, nested mappings, and dynamic arrays.

The shadow view is never treated as independent truth. It is constrained by projection relations back to raw slots. This is essential because Ethereum storage semantics remain slot-based even when higher-level structure is inferred.

6.2 Trace tuples

When a memory hashing site yields a recoverable preimage, the engine records

$$\tau = \langle p, h, n, \mathsf{pc} \rangle, \tag{6.2}$$

where p is the symbolic preimage vector, h is the symbolic hash term, n is the traced byte width, and pc is the program counter at the hash site.

The trace log is therefore a finite sequence

$$\mathcal{T} = (\tau_0, \tau_1, \ldots, \tau_{m-1}). \tag{6.3}$$

The structure inference procedure is a partial classifier

$$\mathsf{Infer} : \mathcal{T}^* \times \tau \to \{\mathsf{FlatMapping}(s), \mathsf{NestedMapping}(s, d), \mathsf{DynamicArray}(s), \mathsf{Unknown}\}. \tag{6.4}$$

6.3 Size-specialized uninterpreted functions

Bit-precise Keccak is too expensive for the intended workflow, so the repository uses size-specialized uninterpreted functions:

$$K_{32} : \mathbb{W}_{256} \to \mathbb{W}_{256}, \tag{6.5}$$

$$K_{64} : \mathbb{W}_{256} \times \mathbb{W}_{256} \to \mathbb{W}_{256}, \tag{6.6}$$

$$K_{96} : \mathbb{W}_{256} \times \mathbb{W}_{256} \times \mathbb{W}_{256} \to \mathbb{W}_{256}, \tag{6.7}$$

$$K_{128} : \mathbb{W}_{256} \times \mathbb{W}_{256} \times \mathbb{W}_{256} \times \mathbb{W}_{256} \to \mathbb{W}_{256}. \tag{6.8}$$

For a preimage composed of $k \in \{1, 2, 3, 4\}$ 256-bit words,

$$\mathsf{Hash}(p_1, \ldots, p_k) = \begin{cases} K_{32}(p_1) & \text{if } k = 1, \\ K_{64}(p_1, p_2) & \text{if } k = 2, \\ K_{96}(p_1, p_2, p_3) & \text{if } k = 3, \\ K_{128}(p_1, p_2, p_3, p_4) & \text{if } k = 4. \end{cases} \tag{6.9}$$

Unexpected widths fall back to a fresh symbolic constant rather than to an unsoundly reused hash term.

6.4 Lazy injectivity

The repository does not globally assert full injectivity of Keccak UFs. Instead, it uses a local projection discipline that preserves parent-child consistency within traced storage chains. In abstract form:

$$h_c = h'_c \Longrightarrow h_p = h'_p, \tag{6.10}$$

where h_c, h'_c are child-hash terms and h_p, h'_p are the projected parent-hash terms in the same trace chain. This is weaker than cryptographic injectivity and stronger than totally unconstrained hashing.

Definition 6.1 (Soundness envelope of the Keccak abstraction)**.** *The Keccak abstraction preserves equality, disequality, and locally traced projection relationships among bounded preimages. It does not prove collision resistance, preimage resistance, or completeness for untraced hash behaviors.*

Remark 6.2. Definition 6.1 is the right level of claim for the implementation. Anything stronger would be deceptive; anything weaker would fail to explain why structured storage recovery works at all.

7 Target Hydration and Bytecode Slicing

7.1 Hydration function

The symbolic engine is only as useful as the target context it receives. DarkSolver therefore front-loads a hydration phase

$$\mathsf{Hydrate} : (\mathcal{A}, \Gamma) \to \Theta. \tag{7.1}$$

The hydration output contains bytecode, account info, scanned storage slots, attacker-side token balance scaffolding, function selectors, NFT callback selectors, statically recognized dead-end PCs, and dependency contexts. The target context is stored thread-locally during objective execution so selector scans and related queries can reuse it without rehydration.

7.2 Strict empty-state classification

The strict zero-state marker from definition 3.1 can be written explicitly as

$$\mathsf{ZERO_STATE}(\Theta) \Longleftrightarrow (\mathsf{code} = \varnothing) \wedge (\mathsf{bal}_{\text{remote}} = 0) \wedge (\mathsf{scan_ok} = \mathsf{true}) \wedge (\mathsf{nonzero_slots} = \varnothing). \tag{7.2}$$

If the storage scan fails, ZERO_STATE is false. This matters because a permissive empty-state default would manufacture false positives in liquidity and token-balance objectives.

7.3 Selector and bytecode slicing

Hydration also constructs a bytecode slice

$$\mathsf{slice}(B) = \langle \mathsf{sel}(B), \mathsf{nft_callbacks}(B), \mathsf{dead}(B) \rangle, \tag{7.3}$$

where $\mathsf{sel}(B)$ is the set of state-changing selectors inferred from bytecode scanning, $\mathsf{nft_callbacks}(B)$ is the NFT callback slice, and $\mathsf{dead}(B)$ is the set of dead-end program counters discovered by static heuristics.

Hydration caches these slices and may reuse similar slices via SimHash-based classification. The point is not perfect bytecode clustering; the point is to avoid repeated bytecode rescans when enough structural similarity exists to justify reuse.

7.4 Proxy and dependency bounds

The public build uses explicit finite bounds during hydration. Representative defaults in the current repository include:

Quantity	Default	Role
proxy resolution max depth	3	bound nested proxy expansion
diamond max facets	12	bound facet hydration fan-out
storage scan limit	1024	bound direct slot enumeration
deep-scan preloader storage limit	16,384	bound larger preload campaigns

These constants are not merely implementation conveniences. They define the semantic horizon of the hydrated target. A deeper proxy graph or a larger hidden storage footprint may exist on-chain without entering the public workflow's model.

8 Objective Synthesis

8.1 Objective templates

An instantiated objective is best understood as a structured query generator

$$o = \langle \mathsf{name}, \Phi_o, \mathsf{witness}_o, \mathsf{policy}_o \rangle, \tag{8.1}$$

where Φ_o is the symbolic formula family generated for the target, $\mathsf{witness}_o$ explains how solver models are reified into witness parameters, and policy_o carries auxiliary bounds or metadata.

The repository's objective catalog is intentionally protocol-aware. Publicly visible families include generic profit search, AMM price impact, PSM draining, collateral-factor lag, TWAP/oracle manipulation, liquidation spirals, governance exploitation, weak randomness, read-only reentrancy, delegatecall storage clash, vault inflation, share-rounding griefing, and composite risk synthesis. When deep scan is enabled, the catalog extends to deeper invariant analysis, reentrancy search, redemption arbitrage, dust bad debt creation, commit-reveal bypass, gambling-scanner logic, and related families.

8.2 Schedule hints and target-specific filtering

Objective generation depends on chain-aware hints. Fast chains are recognized by low block times, which influences deep-scan defaults, pseudo-randomness bounds, and time-drift windows. Objective families are then filtered by allowlists, denylists, and caps:

$$\mathcal{O}' = \mathrm{Cap}_N \Big(\mathrm{Deny}_D \left(\mathrm{Allow}_A(\mathcal{O}) \right) \Big). \tag{8.2}$$

This composition should be read operationally:

- Allow_A retains only objective names matching explicit allow filters when such filters are present;
- Deny_D removes objective names matching deny filters;
- Cap_N truncates the remaining family to a finite target-specific cap.

8.3 Semantic role of objective families

The catalog performs two jobs simultaneously.

1. It reduces search space by encoding *what counts as a relevant exploit hypothesis* for the target.
2. It increases semantic specificity by attaching domain constraints that generic opcode search would never infer, such as reserve-product or collateral-ratio relationships.

In that sense, DarkSolver is closer to a family of targeted proof obligations than to a single monolithic vulnerability predicate.

9 Global Invariant Gates

9.1 Overview

Satisfiability alone is not a sufficient admission criterion. A candidate witness must also satisfy the repository's global invariant gate:

$$\mathsf{Gate} = G_{\mathsf{solv}} \wedge G_{\mathsf{price}} \wedge G_{\mathsf{k}}. \tag{9.1}$$

These gates are especially important for AMM-like objectives, where a naive symbolic model can satisfy purely algebraic path constraints while violating obvious economic sanity.

9.2 Solvency gate

Let $a^\star$ be the canonical attacker address and ℓ the flash-loan principal. If $\mathsf{bal}_\Sigma(a^\star)$ is the final attacker ETH balance term in the symbolic machine, then

$$G_{\mathsf{solv}}(\Sigma, \ell) \iff \mathsf{bal}_\Sigma(a^\star) \geq \ell. \tag{9.2}$$

This is intentionally strict. The gate does not accept a candidate that cannot at least service the modeled principal.

9.3 112-bit reserve normalization

The invariant layer manipulates packed AMM reserves that occupy 112-bit subwords in a 256-bit slot. The normalization operator is:

$$\mathrm{norm}_{112}(x) = \mathsf{zeroExtend}_{144}(x_{111:0}). \tag{9.3}$$

Reserve positivity and finite-width checks are then:

$$G_{\mathrm{finite}}(r) \iff r_{255:112} = 0, \tag{9.4}$$

$$G_{\mathrm{positive}}(r) \iff r > 0. \tag{9.5}$$

9.4 Price-sanity gate

Let (r_0, r_1) be baseline reserves and (r'_0, r'_1) the candidate reserves after the symbolic trace. The implementation compares cross-multiplied ratios in lifted 512-bit space. For a tolerance β basis points:

$$G_{\mathrm{ratio},\beta}(r'_1, r'_0, r_1, r_0) \iff \max\big(\,\mathrm{lift}_{256\to512}(\mathrm{norm}_{112}(r'_1))\,\mathrm{lift}_{256\to512}(\mathrm{norm}_{112}(r_0)), \mathrm{lift}_{256\to512}(\mathrm{norm}_{112}(r_1))\,\mathrm{lift}_{256\to512} \tag{9.6}$$

The full price-sanity gate over all tracked pairs is

$$G_{\mathsf{price}}(\Sigma) \iff \bigwedge_{p\in\mathrm{dom}(\mathsf{res})} \Big(G_{\mathrm{finite}}(r'_{0,p}) \wedge G_{\mathrm{finite}}(r'_{1,p}) \wedge G_{\mathrm{positive}}(r'_{0,p}) \wedge G_{\mathrm{positive}}(r'_{1,p}) \wedge G_{\mathrm{ratio},\beta_p}(r'_{1,p}, r'_{0,p}, r_{1,p}, r_{0,p})\Big). \tag{9.7}$$

9.5 Reserve-product gate

To avoid modular artifacts, reserve-product comparisons are also lifted:

$$G_{\mathsf{k}}(\Sigma) \iff \bigwedge_{p\in\mathrm{dom}(\mathsf{res})} \mathrm{lift}_{256\to512}(\mathrm{norm}_{112}(r'_{0,p}))\cdot\mathrm{lift}_{256\to512}(\mathrm{norm}_{112}(r'_{1,p})) \geq \mathrm{lift}_{256\to512}(\mathrm{norm}_{112}(r_{0,p}))\cdot\mathrm{lift}_{256\to512}(\mathrm{no} \tag{9.8}$$

9.6 Admission

For an objective-generated witness w, symbolic admission can therefore be stated as

$$\mathsf{Admit}_{\mathrm{sym}}(w) \iff \mathsf{SAT}(\Phi_w) \wedge \mathsf{Gate}(w). \tag{9.9}$$

Equation (9.9) is a necessary but not always sufficient criterion for publication-grade evidence, because replay and curation policy may impose stricter requirements.

10 Parallel Execution and Determinism

10.1 Independent solver contexts

Each objective is solved in its own blocking worker and its own Z3 context. This is both a technical necessity and a semantic convenience. It is a necessity because Z3 contexts are not freely shareable

across threads. It is a convenience because it prevents objective-local search state from leaking across objectives.

Let $\mathcal{O}' = \{o_1, \ldots, o_n\}$. The detailed runner computes

$$\mathsf{Run}(\mathcal{O}', B, \Theta, \tau_{\text{tot}}) = (\mathcal{R}_{\mathsf{Status}}, \mathcal{F}), \tag{10.1}$$

where $\mathcal{R}_{\mathsf{Status}}$ is the status-record set and $\mathcal{F}$ is the set of satisfiable findings.

10.2 Status semantics

Each objective yields a status in

$$\mathsf{Status} \in \{\mathsf{SAT}, \mathsf{UNSAT}, \mathsf{TIMEOUT}, \mathsf{PANIC}\}. \tag{10.2}$$

These should be read precisely.

- SAT: the objective produced witness parameters.
- UNSAT: the objective completed without producing witness parameters.
- PANIC: the objective panicked and the panic payload was captured.
- TIMEOUT: the total audit timeout expired before that objective completed, so the runner marks the objective as timed out and aborts pending work on a best-effort basis.

10.3 Determinism boundary

DarkSolver deliberately makes the following quantities stable or nearly stable.

1. Solver parameterization uses fixed seeds and explicit budgets.
2. The detailed runner sorts status records by objective name before returning them.
3. Objective allowlist/denylist/cap logic is deterministic for fixed inputs.

It does *not* claim stable completion order of parallel workers or stable ordering of simple finding vectors when those depend on task completion order. The right mathematical statement is therefore *set-level determinism under fixed solver semantics*, not sequence-level determinism of every emitted list.

10.4 Fail-closed worker handling

If any worker panics or is cancelled in the basic parallel runner, the entire run returns an error instead of emitting partial findings. The code therefore implements the following policy:

$$\exists i \,.\, \mathsf{worker_failed}(o_i) \Longrightarrow \mathsf{Run}_{\text{basic}}(\mathcal{O}', B, \Theta) = \mathsf{Err}. \tag{10.3}$$

This is a meaningful semantic commitment: the repository would rather lose evidence than mix trustworthy and untrustworthy objective outcomes in the same result set.

11 Replay Validation Semantics

11.1 Repository position

The replay/verifier layer is implemented in the repository and is suitable for curated evidence, but it should be described carefully. The default public single-target command path emphasizes solving and objective status. Replay validation is therefore best viewed as a *companion semantics for witness curation*, not as a claim that every CLI path runs a mandatory replay phase before returning.

11.2 Replay input and execution

Given a witness w, a chain environment Γ, and an attacker address $a^\star$, the verifier constructs a replay report

$$\mathsf{Replay}(w, \Gamma, a^\star) = \rho. \tag{11.1}$$

Operationally, the verifier:

1. initializes a forked database, optionally pinned to a historical block;
2. reads the attacker's initial ETH and tracked-token balances;
3. applies any modeled flash-loan inventory to the fork state;
4. groups witness steps by block offsets;
5. executes each step against `revm`, advancing block context when offsets require it;
6. records the first failing step, revert payload, halt reason, and gas use if execution does not remain successful;
7. re-reads ETH and tracked-token balances after execution;
8. computes a mark-to-market report over initial and final holdings.

The current per-step gas limit in the verifier is 2,000,000 gas. This is another explicit bound rather than a hidden assumption.

11.3 Report structure

The replay report contains at least the following fields:

$$\rho = \langle \mathsf{success}, \mathsf{Profit}, \mathsf{gas_used}, \mathsf{failed_step}, \mathsf{initial_eth}, \mathsf{final_eth}, \mathsf{token_deltas}, \mathsf{initial_value}, \mathsf{final_value}, \mathsf{gas_cost}, \mathsf{pric} \tag{11.2}$$

This report is richer than a Boolean replay verdict. It is designed to support downstream policies that care about profitability, asset-coverage quality, and stale valuation inputs.

11.4 Mark-to-market valuation

Let U be the set of tracked tokens. For a token $u \in U$, let $\mathsf{bal}_u^{(t)}(a^\star)$ be the attacker's token balance at time $t \in \{0, 1\}$, $\mathsf{dec}(u)$ its decimals, and $\mathsf{price}(u)$ its ETH price proxy or override. Then

$$\mathsf{Val}_t(a^\star) = \mathsf{ETH}^{(t)}(a^\star) + \sum_{u \in U} \frac{\mathsf{bal}_u^{(t)}(a^\star) \cdot \mathsf{price}(u)}{10^{\mathsf{dec}(u)}}. \tag{11.3}$$

Let $\mathsf{basefee}$ be the observed base fee and tip the configured priority fee. The estimated gas cost is

$$\mathsf{GasCost} = (\mathsf{basefee} + \mathsf{tip}) \cdot \mathsf{gas_used}. \tag{11.4}$$

With a configured minimum margin $m_{\min}$, profitability is:

$$\mathsf{Profit}(\rho) \iff \mathsf{Val}_1(a^\star) > \mathsf{Val}_0(a^\star) + \mathsf{GasCost} + m_{\min}. \tag{11.5}$$

11.5 Replay-based reportability

Because different consumers may want different thresholds, the clean formulation is policy-parametric:

$$\mathsf{Reportable}_\Pi(w, \rho) \iff \mathsf{Admit}_{\mathrm{sym}}(w) \wedge \Pi(\rho). \tag{11.6}$$

A typical strict publication policy is

$$\Pi_{\mathrm{strict}}(\rho) \iff \rho.\mathsf{success} \wedge \rho.\mathsf{Profit} \wedge (\rho.\mathsf{stale} = 0). \tag{11.7}$$

Equation (11.7) is not hard-coded as the only admissibility rule in the repository; it is a principled example of how the verifier data can be consumed.

12 Correctness Posture

This section states concrete propositions the current implementation actually supports. They are intentionally narrower than full language-level soundness theorems.

Proposition 12.1 (Width-preserving arithmetic)**.** *If symbolic operands are represented in $\mathbb{W}_{256}$, then arithmetic performed by the core symbolic engine remains within $\mathbb{W}_{256}$, and comparisons that must avoid wrap-around are explicitly lifted into $\mathbb{W}_{512}$.*

Proof sketch. The symbolic engine constructs arithmetic terms using Z3 bit-vector operators over fixed widths. The concrete embedding of $U256$ words is a total concatenation of four 64-bit limbs, so the symbolic sort is exactly 256 bits. When the invariant layer compares reserve products or ratio numerators, it first applies zero-extension into 512 bits before multiplying. Because the compared reserve quantities are at most 112 bits after normalization, their products fit comfortably within the 512-bit domain and therefore do not wrap. □

Proposition 12.2 (Strict zero-state classification)**.** *The hydration layer cannot classify a target as* ZERO_STATE *unless bytecode is empty, remote balance is zero or absent, storage scanning succeeded, and no non-zero storage slots were found.*

Proof sketch. The implementation computes the zero-state flag by conjunction of exactly those checks. In particular, storage scan failure returns false rather than "unknown but probably empty." Therefore every positive ZERO_STATE classification is justified by explicit evidence, while some true empty states may remain unclassified if hydration is incomplete. □

Proposition 12.3 (Fail-closed worker semantics)**.** *In the basic parallel objective runner, if any worker panics or is cancelled, the run returns an error and does not emit a trusted finding set.*

Proof sketch. The runner gathers worker results through a join set. Panic and cancellation outcomes are collected as worker failures. If any such failure occurs, the runner returns an error after logging the details, instead of returning accumulated findings. Therefore the basic runner never conflates partial success with trustworthy aggregate success. □

Proposition 12.4 (Exact reserve-product comparison after normalization)**.** *For normalized reserve words $x, y, x', y' \in \mathbb{W}_{112}$, the invariant check*

$$\mathrm{lift}_{256\to512}(x)\,\mathrm{lift}_{256\to512}(y) \geq \mathrm{lift}_{256\to512}(x')\,\mathrm{lift}_{256\to512}(y')$$

is an exact arithmetic comparison, not a modular approximation.

Proof sketch. Each normalized reserve fits in 112 bits. The product of two 112-bit values fits in 224 bits. Zero-extending into 512 bits before multiplication therefore preserves the exact integer product. No wrap-around can occur within 512 bits, so the comparison is exact for the normalized reserve domain used by the gate. □

12.1 Non-claims

The repository does *not* currently justify the following stronger claims.

1. Completeness over all selectors, all paths, or all objective families.
2. Cryptographically faithful reasoning about Keccak.
3. Sound profitability under complete market microstructure and slippage modeling.
4. Stable ordering of all emitted findings under parallel completion races.
5. Full replay validation as an unconditional property of every public command path.

This distinction between propositions and non-claims is not academic hygiene for its own sake. It is necessary to keep the paper aligned with the repository's public behavior.

13 Complexity and Resource Model

13.1 Search complexity

The underlying satisfiability problem is, in the worst case, as hard as the objective-generated `QF_ABV` formulas permit. Path explosion, selector fan-out, and protocol-specific side conditions all contribute.

A fully general complexity theorem would not be useful because the repository is not intended to run unconstrained search. The right complexity statement is conditional on the bounding regime.

Let n be the number of objectives after allow/deny/cap filtering, s_i the selector family relevant to objective o_i, d_i the symbolic step bound embedded in that objective, and h_i the hydration footprint visible to that objective. Then a coarse operational upper bound is

$$T_{\text{run}} \lesssim \sum_{i=1}^{n} T_{\text{solve}}\big(|s_i|, d_i, |h_i|, |\Phi_{o_i}|\big), \tag{13.1}$$

where T_{solve} is additionally clipped by solver timeout and resource limits.

13.2 Implemented bounds

The public repository uses several explicit bounds to make equation (13.1) meaningful in practice.

Bound	Current default	Effect
per-solver timeout	60,000 ms	clips individual solver contexts
solver rlimit	200,000,000	clips low-level resource usage
telemetry solve budget target	1,800 ms	operator-facing pressure target
global price sanity tolerance	250,000 bps	clips admissible reserve drift
per-step replay gas limit	2,000,000 gas	clips replay execution cost
proxy depth	3	clips recursive hydration
diamond facet count	12	clips facet fan-out
direct storage scan	1024 slots	clips immediate storage enumeration
deep preload storage cap	16,384 slots	clips heavier preloading

13.3 Operational interpretation

These bounds imply three practical consequences.

1. The workflow is engineered to degrade by timeout or fail-closed rejection, not by silent widening.
2. Complexity is partly shifted from path depth to objective design: a better objective family can dominate a deeper generic search.
3. Performance claims must be interpreted as contingent on the chosen bounds, not as asymptotic guarantees about unbounded symbolic execution.

14 Limitations and Threats to Validity

14.1 Boundedness

The strongest limitation is also the central engineering choice: bounded search. If an exploit requires selectors outside the scanned slice, proxy expansion beyond the configured depth, storage dependencies outside the scan horizon, or a longer multi-step trace than the relevant objective considers, the repository may miss it.

14.2 Hash abstraction

The Keccak abstraction is deliberately incomplete. Size-specialized uninterpreted functions preserve structured dependence for traced preimages, but they do not model collisions, preimage structure outside recorded traces, or exact digest-level bit arithmetic. This is acceptable for storage-structure recovery; it is not enough for cryptographic proof obligations.

14.3 Hydration dependence

The quality of the symbolic model depends on RPC hydration. Bytecode, account info, storage slots, proxy metadata, and dependency contexts are all hydrated from chain state. Weak or rate-limited RPC responses can degrade coverage or force fail-closed behavior. This is why the repository includes separate RPC benchmarking and operational telemetry tooling.

14.4 Replay coverage

Replay validation is richer than satisfiability, but it is still a bounded simulation over a forked state. It does not inherently model builder behavior, mempool competition, or all real-world execution externalities. Even the profitability layer depends on price proxies, configured overrides, and stale-price thresholds.

14.5 Policy dependence

What counts as "reportable" is downstream-policy dependent. Some operators may accept $\rho.\mathsf{success}$ alone. Others may require strict profitability, no stale prices, or even external human confirmation. This is why $\mathsf{Reportable}_{\Pi}$ is parameterized in equation (11.6).

14.6 Determinism limits

Although solver seeds and status-record ordering are stabilized where practical, parallel execution and remote hydration mean that some operational outputs remain sensitive to environment and runtime timing. The correct claim is reproducibility under controlled conditions, not absolute environment-free determinism.

15 Conclusion

DarkSolver is best understood as a bounded formal-analysis primitive for EVM business-logic search. Its value does not come from pretending to solve arbitrary contract verification in one stroke. It comes from an explicit combination of ingredients that fit the actual repository:

1. width-correct symbolic EVM semantics in `QF_ABV`;
2. target hydration that exposes selectors, dependencies, and strict empty-state classification;
3. protocol-aware objective synthesis instead of undifferentiated path search;

4. triple-gate invariant filtering to suppress economically nonsensical models;
5. fail-closed parallel execution and structured status reporting;
6. replay-oriented report construction for stricter downstream curation.

That combination is rigorous enough to merit whitepaper treatment precisely because it does not overclaim. The repository's mathematics are strongest where its implementation is most explicit: boundedness, width preservation, structured storage inference, invariant gating, and evidence-oriented workflow design. Those are the claims that should survive close comparison between the PDF and the codebase.

References

[1] Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger. Yellow Paper, originally released 2014 and maintained in later revisions. https://ethereum.github.io/yellowpaper/paper.pdf

[2] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Proceedings of TACAS 2008*, pages 337–340, 2008.

[3] Clark Barrett, Aaron Stump, and Cesare Tinelli. The SMT-LIB standard: Version 2.0. Technical report, 2010. https://smtlib.cs.uiowa.edu/

[4] James C. King. Symbolic execution and program testing. *Communications of the ACM*, 19(7):385–394, 1976.

[5] Cristian Cadar and Koushik Sen. Symbolic execution for software testing: Three decades later. *Communications of the ACM*, 56(2):82–90, 2013.

[6] Bluealloy contributors. revm: Rust Ethereum virtual machine implementation. Software project. https://github.com/bluealloy/revm

[7] Microsoft Research. Z3 Guide. Documentation resource. https://microsoft.github.io/z3guide/

[8] Alloy contributors. Alloy: modular Rust toolkit for Ethereum. Software project. https://github.com/alloy-rs/alloy